



清华大学

综合论文训练

保真度感知的自适应量子路由协议研究 与仿真实现

系 别： 计算机科学与技术系

专 业： 计算机科学与技术

姓 名： 王思图

指导教师： 施新刚，副研究员

二〇二六年五月

摘 要

量子互联网的核心任务是在多跳网络中为上层应用分发高保真纠缠资源。与经典最短路径不同，量子路由的路径质量不能只由跳数、成功率或某个静态阈值刻画，而必须显式考虑每一跳链路的初始保真度、建立时间以及量子存储器相干时间。特别是在地面光纤与卫星短链路并存的混合网络中，较早建立的链路需要在量子存储器中等待更慢的链路完成，等待造成的退相干会使局部看似更优的前缀在接入相同后缀后失去优势。

本研究围绕 qns-3 平台实现并分析一种面向端到端保真度的量子网络层协议。我们在路由标签中显式保留整条前缀的链路段信息，使用 $T_{\max}$ 对齐所有链路的就绪时间，并在等待退相干后递归执行 Bell 对角态交换，从而得到可直接用于路径比较的预测保真度。基于这一非加性度量，系统实现了单标签 Dijkstra 基线与 Sliced Dijkstra 多标签搜索，其中后者通过保留不同 $T_{\max}$ 切片降低前缀陷阱导致的误剪枝风险。

当前版本的工作重点是把建模、算法语义与非保序性证明对齐到实现本身，因此实验章节仍保留为后续补充的占位内容。后续将基于真实拓扑与混态重跑进一步评估切片宽度和标签保留上限对路由结果的影响。

关键词：量子网络；网络层协议；路由算法；保真度优化；qns-3

目 录

Part I 量子路由理论研究	1
第 1 章 量子网络物理建模.....	2
1.1 多跳量子中继网络的物理模型	2
1.1.1 节点与链路参数	2
1.1.2 纠缠交换协议的时序模型	3
1.2 端到端纠缠保真度的解析公式	3
1.2.1 退相干模型	3
1.2.2 多跳端到端保真度	4
第 2 章 量子网络路由协议综述.....	6
2.1 量子路由问题的基本约束	6
2.2 基于保真度乘积的启发式路由	6
2.3 当前实现采用的路由基线与切片算法	7
2.4 Caleffi 的最优纠缠率路由.....	8
2.5 相关工作	8
2.6 本章小结	9
第 3 章 量子路由度量的非保序性分析.....	10
3.1 路径标签与当前路由度量	10
3.2 度量的非保序性	11
3.2.1 保序性的定义	11
3.2.2 一个构造性反例	11
3.2.3 一个具体数值例子	12
3.3 Dijkstra 与 Sliced Dijkstra	13
3.3.1 单标签 Dijkstra 作为基线	13
3.3.2 切片多标签搜索	13
3.3.3 算法流程	13
3.3.4 为什么切片能缓解反转	14
3.3.5 算法保证与限制	15
3.4 证明与实现对照	15
3.5 实验设计占位	16

Part II 量子网络模拟器实现 **17**

第 4 章 量子-经典混合仿真平台 qns-3 的架构设计18

4.1 总体设计目标与分层结构18

4.2 离散事件驱动的量子-经典协同仿真18

4.3 ExaTN 密度矩阵计算后端.....19

4.4 量子物理层与存储器退相干模型20

4.5 量子链路层：纠缠对分发与状态管理21

4.6 量子网络层与路由协议21

4.7 经典信令信道的建模方式22

参考文献.....23

Part I

量子路由理论研究

第 1 章 量子网络物理建模

本章建立当前路由实现所采用的路径级物理模型。与将路径压缩成单一累计代价的旧式建模不同，当前实现会把一条候选路径的每一段链路都显式保存在路由标签中，并在扩展时重新计算整条前缀的 $T_{\max}$ 、等待退相干和 Bell 对角态交换结果。后续章节关于非保序性、DijkstraRoutingProtocol 基线和 SlicedDijkstraRoutingProtocol 的分析，都以本章给出的这套路径语义为基础。

1.1 多跳量子中继网络的物理模型

1.1.1 节点与链路参数

考虑由若干量子中继节点构成的一般网络图。对一条候选路径

$$p = (v_0, v_1, \dots, v_h),$$

其第 j 段链路

$$e_j = (v_{j-1}, v_j)$$

在当前实现中不会被压缩成一个标量，而是作为一条完整的路径段保存在路由标签里。每一段链路都记录如下物理参数：

- 左右端节点名；
- 初始保真度 initialFidelity；
- 噪声族 noiseFamily；
- 量子链路建立时间 setupTimeMs；
- 左右端节点的相干时间 τ 。

这些参数来自链路层和物理层的联合建模。initialFidelity 由信道噪声和纠缠生成不完美性共同决定；setupTimeMs 优先读取 quantumSetupTimeMs，若未给出则按当前实现里的 UseCurrentLinkDelay、latency 与 classicalControlDelayMs 顺序回退； τ 则由量子存储器所在节点决定。需要强调的是，当前 successRate 只作为链路能否扩展的门槛使用：当 isAvailable 为假或 successRate

≤ 0 时, 该边不会进入候选; 但一旦候选可扩展, `successRate` 本身不参与 `predicted_fidelity` 的计算。

对应地, 单跳链路的初始 Bell 态仍按 Werner 态参数化:

$$\rho_i^{(0)} = F_i^{(0)} |\Phi^+\rangle\langle\Phi^+| + \frac{1 - F_i^{(0)}}{3} (|\Phi^-\rangle\langle\Phi^-| + |\Psi^+\rangle\langle\Psi^+| + |\Psi^-\rangle\langle\Psi^-|) \quad (1.1)$$

其中 $|\Phi^\pm\rangle, |\Psi^\pm\rangle$ 为四个 Bell 基态。

1.1.2 纠缠交换协议的时序模型

当前实现采用的不是“逐跳串行建链”的时序, 而是“整条候选路径并行开始、按最慢链路对齐”的时序:

1. 路径上的各段链路在同一轮评估中同时开始建立纠缠;
2. 每段链路的就绪时间由其 `setupTimeMs` 给出;
3. 整条路径的统一对齐时刻定义为

$$T_{\max}(p) = \max_{1 \leq j \leq h} T_j, \quad T_j \equiv \text{setupTimeMs}(e_j); \quad (1.2)$$

4. 更快完成的链路必须等待较慢链路, 于是第 j 段的等待时间为

$$W_j(p) = T_{\max}(p) - T_j; \quad (1.3)$$

5. 交换操作本身在路由 `metric` 中视为瞬时完成, 不额外消耗建链时间。

因此, 当前模型中的退相干不是沿着经典信令逐跳累积出来的, 而是由路径内部的时间错配一次性引入, 然后再通过后续交换逐段传播到端到端保真度。这个处理方式也是后续章节中 $T_{\max}$ 切片的根本原因。

1.2 端到端纠缠保真度的解析公式

1.2.1 退相干模型

qns-3 当前的存储退相干模型并不直接把 Bell 保真度乘上一个指数因子, 而是对单个存储量子比特施加相位翻转信道

$$\mathcal{E}_\lambda(\rho) = \frac{1 + \lambda}{2} \rho + \frac{1 - \lambda}{2} (Z \rho Z), \quad \lambda(\Delta t, \tau) = e^{-\Delta t/\tau}, \quad (1.4)$$

其中 Δt 为等待时间, τ 为该量子比特所在节点的相干时间。对一对以式 (1.1) 初始化的 Bell 对, 若左右两端量子比特分别等待 $\Delta t_L, \Delta t_R$, 则其状态仍保持 Bell 对角形式:

$$\rho_i(\Delta t_L, \Delta t_R) = a_i |\Phi^+\rangle\langle\Phi^+| + b_i |\Phi^-\rangle\langle\Phi^-| + c_i |\Psi^+\rangle\langle\Psi^+| + d_i |\Psi^-\rangle\langle\Psi^-|, \quad (1.5)$$

其中

$$\begin{aligned}
q_i &\triangleq \frac{1 - F_i^{(0)}}{3}, \\
\Lambda_i &\triangleq \exp\left(-\frac{\Delta t_L}{\tau_i} - \frac{\Delta t_R}{\tau_{i+1}}\right), \\
a_i &= \frac{2F_i^{(0)} + 1 + \Lambda_i(4F_i^{(0)} - 1)}{6}, \\
b_i &= \frac{2F_i^{(0)} + 1 - \Lambda_i(4F_i^{(0)} - 1)}{6}, \\
c_i &= d_i = q_i.
\end{aligned} \tag{1.6}$$

因此，该链路在等待后的 Bell 保真度应写为

$$F_i(\Delta t_L, \Delta t_R) = \langle \Phi^+ | \rho_i(\Delta t_L, \Delta t_R) | \Phi^+ \rangle = a_i, \tag{1.7}$$

而不是简单的 $F_i^{(0)} e^{-\Delta t/\tau}$ 。

1.2.2 多跳端到端保真度

设 Bell 对角态的 Bell 权重向量记为

$$w = (a, b, c, d), \quad a + b + c + d = 1, \tag{1.8}$$

其中 a 即相对于 $|\Phi^+\rangle$ 的 Bell 保真度。对两端等待导致的相位翻转退相干，可定义等待算子

$$\mathcal{M}_\Lambda(a, b, c, d) = \left(\frac{1+\Lambda}{2}a + \frac{1-\Lambda}{2}b, \frac{1-\Lambda}{2}a + \frac{1+\Lambda}{2}b, \frac{1+\Lambda}{2}c + \frac{1-\Lambda}{2}d, \frac{1-\Lambda}{2}c + \frac{1+\Lambda}{2}d \right), \tag{1.9}$$

其中 Λ 由式 (1.6) 给出。

若两段相邻路径的端到端态分别为 $w = (a, b, c, d)$ 与 $w' = (a', b', c', d')$ ，则一次理想纠缠交换后的输出仍为 Bell 对角态，其 Bell 权重满足

$$S(w, w') = (aa' + bb' + cc' + dd', ab' + ba' + cd' + dc', ac' + ca' + bd' + db', ad' + da' + bc' + cb'). \tag{1.10}$$

因此，多跳端到端纠缠保真度的**正确形式**不是标量乘积，而是对各跳 Bell 对角态先施加等待算子 $\mathcal{M}_\Lambda$ ，再递归施加交换算子 S ；最终端到端保真度等于递推结果的第一个分量。

在当前代码实现里，这一过程等价于先对整条路径调用 `EvaluateSegments`，再把返回的 Bell 对角态第一个分量记为 `predicted_fidelity`，同时把 `t_max_ms` 写入路由标签。特别地，在

无额外等待且输入均为 Werner 态的情形，若两条相邻链路初始保真度分别为 F_1, F_2 ，则一次纠缠交换后的 Bell 保真度为

$$F_{\text{swap}} = F_1 F_2 + \frac{(1 - F_1)(1 - F_2)}{3} = \frac{4F_1 F_2 - F_1 - F_2 + 1}{3}, \quad (1.11)$$

而不是简单的 $F_1 F_2$ 。式 (1.7) 和式 (1.11) 正是当前 qns-3 物理层混态建模与理想纠缠交换所对应的闭式表达。

第 2 章 量子网络路由协议综述

本章梳理量子网络路由问题的基本约束，介绍代表性量子路由协议的设计思路，并分析其局限性，为后续章节对路由非保序性的理论分析奠定背景。

2.1 量子路由问题的基本约束

与经典网络中的最短路径或最优流量路由不同，量子网络路由必须同时应对如下物理约束：

1. **纠缠生成的概率性**：量子信道的 Bell 对生成成功率 $p_e \in (0, 1)$ 受限于光子传输损耗、探测器效率和光学对准误差等因素，路径的端到端纠缠建立成功率随跳数指数下降。
2. **量子态的不可克隆性**：量子信息不能被复制（量子不可克隆定理），因此量子路由无法像经典路由那样通过报文重传或多播分发保证可靠性；一旦纠缠测量失败，该量子资源便不可恢复。
3. **保真度衰减**：纠缠对在量子存储器中等待时会因退相干而损失保真度（参见第 1 章）。在本文的路由度量中，这一效应主要表现为不同链路就绪时间造成的 $T_{\max}$ 对齐等待；在完整仿真中，额外经典信令等待也会由物理层退相干模型反映。
4. **存储器相干时间限制**：现有量子中继节点的相干时间 τ 通常在毫秒至秒级 Lin et al. (2024)，超过相干时间的等待将使纠缠资源完全退化；路由协议须在有效的相干时间窗口内完成多跳纠缠交换。

上述约束决定了量子路由不能简单将跳数或带宽作为优化目标，而需要联合考虑链路成功率、保真度衰减和相干时间等量子特征参数。

2.2 基于保真度乘积的启发式路由

一种直觉性的量子路由思路是把路径质量近似写成各链路初始保真度的连乘：若忽略等待退相干和交换过程，端到端保真度可粗略近似为

$$F_{\text{end}}(P) \approx \prod_{e \in P} F_e^{(0)}.$$

在这个非常粗糙的近似下，可以用 $-\log F_e^{(0)}$ 作为链路权重，把路径选择化约为标准最短路径问题。这个思路的意义主要在于提供一个最小复杂度的基线，而不是

作为当前实现中的最终度量。

该近似只有在两个条件同时较强时才勉强成立：

1. 各链路的初始保真度大致独立且已知；
2. 路径内部的等待时间可以忽略，或者所有链路的建立时间都足够接近。

一旦路径内部的建立时间出现明显错配，路径标签就不能再被压缩成一个标量乘积。当前实现采用的正是后一种更完整的路径级语义：整条前缀会在扩展时重新计算 $T_{\max}$ ，因此“局部最优前缀”并不一定是“接入共享后缀后的全局最优前缀”。这也是后续章节讨论 Dijkstra 基线失效的起点。

2.3 当前实现采用的路由基线与切片算法

与上述粗略启发式不同，当前 qns-3 的路由层采用的是路径标签级的精确重算语义。每条 label 都保存完整的链路段列表，扩展时重新评估整条路径的 `predicted_fidelity` 与 `t_max_ms`，因此路径比较不再是单纯的代价累加，而是“前缀重算 + 时间瓶颈比较”。

在这一实现下，`DijkstraRoutingProtocol` 的作用是单标签基线。它每次只保留每个节点当前最优的一条 label，因此如果一个前缀因为局部保真度更高而抢占了名额，另一个在当前节点略差、但 `t_max_ms` 更小的前缀就可能被过早剪枝。换句话说，它在当前度量下并不是“经典加性 Dijkstra 的严格最优版本”，而只是 $K = 1$ 的 best-first baseline。

与之对应，`SlicedDijkstraRoutingProtocol` 是多标签搜索。它允许同一节点保留最多 K 条 label，并且可以沿 $T_{\max}$ 轴按固定宽度分桶：

$$\text{bucket_id}(p) = \left\lfloor \frac{T_{\max}(p)}{\text{BucketWidthMs}} \right\rfloor.$$

当 `UseBuckets = true` 时，不同 bucket 的 label 可以共存；同一 bucket 内则优先保留 `predicted_fidelity` 更高的候选。这样做的目的不是提高单条路径的局部分数，而是尽量保留“快但稍弱”和“慢但更强”的不同前缀切片，从而减少后缀接入后发生排序反转的概率。

当 `UseBuckets = false` 时，算法退化为基于 `predicted_fidelity` 与 `t_max_ms` 的 Pareto 多标签搜索。也就是说，它保留的是关于“端到端保真度更高且时间瓶颈不更差”的一组支配前沿，而不是强行把所有候选压缩成一个标量排名。需要注意的是，切片算法并不是全路径枚举算法；在有限 K 与同桶竞争规则下，它仍可能丢掉未来会变好的前缀，因此本文把它作为缓解非保序剪枝的多标签基线，而不是带严格最优性保证的精确算法。

2.4 Caleffi 的最优纠缠率路由

Caleffi (2017) 在量子中继网络中建立了端到端纠缠生成率 (entanglement generation rate) 的精确随机模型。该工作将原子-光子纠缠生成概率、光子-光子贝尔态测量不完美性、退相干时间等多种物理因素纳入统一的随机分析框架，推导了任意路径的端到端纠缠率闭合表达式，并以最大化纠缠率为路由优化目标。

Caleffi 的核心发现是：纠缠生成率度量具有非保序性 (non-isotonicity) ——在路径拼接时，两条路径的优劣排序可能发生逆转，使得局部最优选择在全局上非最优。这一性质从根本上破坏了标准 Dijkstra 算法的正确性前提：Dijkstra 的贪心选择依赖于路径度量对拼接操作的单调性，而非保序性度量无法满足这一条件。针对该缺陷，Caleffi 提出通过路径枚举算法穷举所有候选路径以求得全局最优解。这一工作是量子路由非保序性问题的早期理论奠基之一，但路径枚举的指数级复杂度限制了其在大规模网络中的实用性。

值得注意的是，Caleffi 所分析的路由度量是**纠缠生成率**，与上节保真度乘积方法的优化目标不同；路由非保序性问题在两种度量体系中均以不同形式存在，而下一章将针对本文当前实现中的 BottleneckFidelityRoutingMetric 给出严格的非保序性理论分析。

2.5 相关工作

在上述主流方法之外，若干研究针对特定场景提出了改进方案。Ali 等人 Ali et al. (2017) 研究了量子网络编码在多播纠缠分发中的最优性条件，为量子组播路由提供了理论依据。Wang 等人 Wang et al. (2025) 基于路由代数框架提出了 K-最优路径路由 (KOP)，从代数结构的角度系统分析了量子路由度量的非保序性，并设计了在代数意义上保证收敛性的修正算法。Sutcliffe 等人 Sutcliffe et al. (2025) 针对多方纠缠分发场景，提出了保真度感知的多路径路由策略，在保证最低保真度阈值的前提下优化分发吞吐量。Shi 和 Qian 提出的 Q-CAST Shi et al. (2020) 则代表了多路径并发和吞吐量优先思路的一个典型分支，可作为本文单标签基线之外的对照背景。

面向卫星量子网络的全球尺度纠缠分发，QuESat Gu et al. (2025) 设计了基于低轨道卫星星座的被动光学量子中继网络，并提出了针对星地混合拓扑的路由策略。该工作的出现使得“高延迟-高保真度”卫星链路与“低延迟-低相干时间”地面节点共存的异质场景变得更加现实，对量子路由算法提出了更高的要求。

2.6 本章小结

现有量子路由协议大致可以分为三类：（1）基于保真度乘积或类似粗粒度度量的启发式最短路径方法；（2）以纠缠率或可达速率为度量的代数分析方法；（3）以多路径并发和吞吐量为中心的协议族。三类方法的共同局限在于：**都没有把当前实现中的路径级 $T_{\max}$ 重算语义作为核心对象来处理**。因此，单标签 Dijkstra 在当前模型下可能过早剪枝，而多标签切片算法则能够更稳妥地保留不同时间前缀的代表性候选，但它本身仍是有限标签近似。下一章将围绕这一点，给出当前度量的非保序性分析以及 Dijkstra 与 Sliced Dijkstra 的对比证明。

第 3 章 量子路由度量的非保序性分析

本章在第 1 章路径级物理模型的基础上，分析当前 qns-3 路由度量为什么不是保序的，并说明单标签 DijkstraRoutingProtocol 为什么会在这种度量下过早剪枝。随后给出 SlicedDijkstraRoutingProtocol 的多标签切片语义，解释它如何通过保留不同 $T_{\max}$ 前缀来缓解排序反转。实验部分暂不报告最终数值，只保留后续验证所需的设计占位。

3.1 路径标签与当前路由度量

当前实现中的路径不是一个单一标量，而是一条完整的 label。对路径

$$p = (v_0, v_1, \dots, v_h),$$

其标签中显式保存了每一段链路的原始参数：

- initialFidelity，表示该段链路的初始 Bell 态保真度；
- noiseFamily，表示该段链路的噪声族；
- setupTimeMs，表示该段链路的就绪时间；
- 左右端节点的相干时间 τ 。

这些段信息会在扩展时被重新扫描，并共同决定当前 label 的两个核心量：

$$T_{\max}(p) = \max_j T_j, \quad F_{\text{pred}}(p) = \text{predicted_fidelity}(p).$$

其中 T_j 是第 j 段链路的 setupTimeMs。当前 metric 的计算不是“在旧分数上加一项”，而是对整条前缀重新执行：

1. 计算整条路径的 $T_{\max}$ ；
2. 令更快的链路等待 $T_{\max} - T_j$ ；
3. 对等待后的 Bell 对角态递归做纠缠交换；
4. 从最终 Bell 对角态的第一个分量读取 predicted_fidelity。

因此，当前 metric 本质上是一个“前缀重算”的路径质量函数，而不是一个可加的边权和。

3.2 度量的非保序性

3.2.1 保序性的定义

若一个路径度量 F 满足

$$F(a) \geq F(b) \implies F(a \oplus c) \geq F(b \oplus c) \quad (3.1)$$

则称它在路径拼接下是保序的。这个性质对经典最短路径算法非常关键，因为它保证了局部最优前缀拼接后仍然不劣于局部次优前缀。

当前实现中的 `predicted_fidelity` 不满足这一性质。原因不是某一条边的局部打分出了错，而是 $T_{\max}$ 会在拼接后重新决定所有已建链路的等待时间，使得一个“当前更好”的前缀在接入共同后缀后，反而更快被退相干拖垮。

3.2.2 一个构造性反例

为了避免把复杂递推写成一个不存在的闭式，我们直接使用当前代码中已经实现的两步演化来构造反例。

先考虑一条单段 Werner 路径。若其初始保真度为 F ，左右两端的相干时间都取 τ ，并且该段在交换前需要等待 Δ ，则其等待后的 Φ^+ 权重为

$$g(F, \Delta; \tau) = \frac{2F + 1 + e^{-2\Delta/\tau}(4F - 1)}{6}. \quad (3.2)$$

这正是 `ApplyPhaseFlipMemoryWait` 在 Werner 输入、对称相干时间和对称等待时间下的特例。可以直接看出，式 (3.2) 随 F 单调递增，随 Δ 单调递减。

再考虑一个固定后缀 x 。若后缀在交换时的 Bell 保真度为 $F_x > 1/4$ ，则 Werner 交换公式

$$S(F_1, F_2) = F_1 F_2 + \frac{(1 - F_1)(1 - F_2)}{3} \quad (3.3)$$

关于每一个自变量都是单调递增的，因为

$$\frac{\partial S}{\partial F_1} = \frac{4F_2 - 1}{3}, \quad \frac{\partial S}{\partial F_2} = \frac{4F_1 - 1}{3},$$

在本节关注的物理区间 $F_1, F_2 > 1/2$ 内，这两个偏导都为正。

于是，只要存在两个前缀 a, b ，使得

$$F_{\text{pred}}(a) > F_{\text{pred}}(b), \quad T_{\text{max}}(a) > T_{\text{max}}(b),$$

并且在接入同一快速后缀 x 后, x 在 $a \oplus x$ 中需要比在 $b \oplus x$ 中等待更长时间, 那么就可能出现

$$S(F_{\text{pred}}(a), g(F_x, \Delta_x^{(a)}; \tau)) < S(F_{\text{pred}}(b), g(F_x, \Delta_x^{(b)}; \tau)),$$

其中 $\Delta_x^{(a)} > \Delta_x^{(b)}$ 。于是由式 (3.3) 推出

$$F_{\text{pred}}(a \oplus x) < F_{\text{pred}}(b \oplus x).$$

下面的数值例子给出一组满足该不等式的参数, 从而完成构造性证明。

3.2.3 一个具体数值例子

为了把上述构造说得更直观, 可以取如下参数:

- 前缀 a : 单段路径, 初始保真度 $F_a = 0.55$, 就绪时间 $t_a = 5$ ms;
- 前缀 b : 单段路径, 初始保真度 $F_b = 0.50$, 就绪时间 $t_b = 0$ ms;
- 共享后缀 x : 单段路径, 初始保真度 $F_x = 0.90$, 就绪时间 $t_x = 0$ ms;
- 所有节点的相干时间都取 $\tau = 20$ ms。

不接后缀时, 显然有

$$F_{\text{pred}}(a) = 0.55 > 0.50 = F_{\text{pred}}(b).$$

接上同一个后缀后, $a \oplus x$ 的 T_{max} 是 5 ms, $b \oplus x$ 的 T_{max} 是 0 ms。于是后缀 x 在 $a \oplus x$ 中需要等待 5 ms, 而在 $b \oplus x$ 中不需要等待。根据式 (3.2),

$$g(0.90, 5; 20) \approx 0.7295, \quad g(0.90, 0; 20) = 0.90.$$

再代入式 (3.3), 得到

$$F_{\text{pred}}(a \oplus x) \approx 0.4418, \quad F_{\text{pred}}(b \oplus x) \approx 0.4667.$$

因此, 尽管 a 在中间节点处更优, 但接入同一后缀后, 它反而输给了 b 。这组数值完成了非保序性的构造性证明, 也解释了单标签 DijkstraRoutingProtocol 会出现过早剪枝的根本原因。

3.3 Dijkstra 与 Sliced Dijkstra

3.3.1 单标签 Dijkstra 作为基线

DijkstraRoutingProtocol 在当前实现中的角色是 $K = 1$ 单标签基线。它为每个节点只保留一条 `incumbent label`, 并始终按 `predicted_fidelity` 选择当前最优前缀; 当分数相同的时候, 再比较 `t_max_ms`, 最后才比较路径长度。

这个策略在加性最短路问题里很自然, 但在当前非保序度量下会产生风险: 某个前缀只要暂时略优, 就可能把另一个更慢但未来更强的前缀挤掉。换句话说, 单标签 Dijkstra 并不是“算法实现不够精细”, 而是它所依赖的局部支配前提已经不成立。

3.3.2 切片多标签搜索

SlicedDijkstraRoutingProtocol 采用的是多标签搜索。它允许同一节点保留最多 K 条 `label`, 并且在 `UseBuckets = true` 时沿 $T_{\max}$ 轴切片:

$$\text{bucket_id}(p) = \left\lfloor \frac{T_{\max}(p)}{\text{BucketWidthMs}} \right\rfloor.$$

这个设计的目标不是简单“保留更多路径”, 而是让不同时间瓶颈的前缀可以共存。这样一来, 快但略弱的前缀和慢但更强的前缀都有机会活到后缀接入之后, 再由真实的路径级重算决定最终胜者。

当 `UseBuckets = false` 时, 算法退化为基于 `predicted_fidelity` 和 `t_max_ms` 的 Pareto 多标签搜索。也就是说, 它不再只看分数高低, 而是同时要求“保真度更高且时间瓶颈不更差”才允许支配关系成立。

3.3.3 算法流程

算法 3.1 给出了当前实现的核心流程。为了突出与普通 Dijkstra 的差异, 伪代码只保留路由搜索本身, 省略拓扑缓存、日志和 `ns-3` 对象管理细节。

其中, `AdmitLabel` 是切片算法的关键。若启用 `bucket`, 则候选 `label` 先按

$$\left\lfloor \frac{T_{\max}}{B} \right\rfloor$$

落入固定时间桶; 同一节点同一桶中只保留 `IsBetter` 意义下最好的代表。若未启用 `bucket`, 则采用 `Dominates` 定义的 Pareto 支配关系: 只有当一个 `label` 在 `predicted_fidelity` 上不低、在 `t_max_ms` 上不高, 并且至少一项严格更好时, 才可以删除另一个 `label`。每次插入后, 同一节点的标签集合都会按 `IsBetter`

算法 3.1 Sliced Dijkstra 的多标签搜索流程

Require: 源节点 s , 目的节点 t , 拓扑 G , 每节点标签上限 K , 桶宽 B

Ensure: 从 s 到 t 的候选路径

```
1:  $L[v] \leftarrow \emptyset$  for all  $v \in G$ 
2:  $l_0 \leftarrow \text{CreateInitialLabel}(s)$ 
3:  $L[s] \leftarrow \{l_0\}$ , 并将  $l_0$  插入按  $\text{predicted\_fidelity}$  排序的优先队列  $Q$ 
4:  $best \leftarrow \emptyset$ 
5: while  $Q$  非空 do
6:    $l \leftarrow Q.\text{pop}()$ 
7:    $u \leftarrow l$  的末端节点
8:   if  $l$  已不在  $L[u]$  中 then
9:     continue
10:  end if
11:  if  $u = t$  then
12:    若  $l$  优于  $best$ , 则  $best \leftarrow l$ 
13:    continue
14:  end if
15:  for 每条可用出边  $(u, v)$  do
16:     $l' \leftarrow \text{ExtendLabel}(l, v)$ 
17:    if  $l'$  被  $\text{AdmitLabel}(v, l')$  接收 then
18:      将  $l'$  插入  $Q$ 
19:    end if
20:  end for
21: end while
22: return  $best$  对应的节点路径
```

排序, 并截断到最多 K 条。

设图中节点数为 $|V|$ 、边数为 $|E|$ 。由于每个节点最多保留 K 条 label, 搜索过程中最多有 $O(K|V|)$ 个活跃 label, 每个活跃 label 至多扩展其出边一次, 因此边扩展次数为 $O(K|E|)$ 。每次扩展需要调用 ExtendLabel 对整条前缀重算 $T_{\max}$ 、等待退相干和 Bell 对角态交换; 若当前前缀长度为 h , 这一部分代价为 $O(h)$ 。此外, AdmitLabel 至多扫描同节点的 K 条 label, 优先队列操作带来 $O(\log(K|V|))$ 的代价。因此, 切片算法相对于单标签基线的额外成本主要来自两个方面: 同一节点保留多个候选, 以及每个候选扩展时无法使用加性增量更新, 只能重算完整前缀。

3.3.4 为什么切片能缓解反转

对当前模型来说, Sliced Dijkstra 的价值在于它不会把所有前缀都压缩成一个 incumbent。只要两个候选前缀落在不同的 $T_{\max}$ 切片里, 它们就可以同时留下来。这样, 当后缀的等待敏感性足够强时, 算法仍然有机会从保留下来的慢前缀和快前缀之间选出真正更优的那一条, 而不是被单标签基线提前锁死。

从这个角度看, Sliced Dijkstra 不是“更复杂的 Dijkstra”, 而是对

非保序度量的一种结构化修正。

3.3.5 算法保证与限制

需要特别说明的是，Sliced Dijkstra 是一种面向当前非保序度量的启发式修正，而不是全局最优路径枚举算法。它相对单标签 Dijkstra 的优势在于扩大了同一节点可保留的前缀集合，尤其允许不同 $T_{\max}$ 切片的候选共存；但只要 K 有限，或者同一 bucket 内仍然只保留一个局部代表，它就仍然可能丢弃某条未来会变好的前缀。

更形式化地说，若希望切片算法严格不劣于单标签 Dijkstra，至少需要满足如下条件之一：

1. 切片算法在每个节点保留了单标签 Dijkstra 最终路径所需的所有前缀；
2. 或者，被保留的前缀能够在任意后缀扩展下支配被删除的前缀。

当前实现并不保证这两个条件。特别是在同一节点、同一 $T_{\max}$ bucket 内，局部 predicted_fidelity 更高的 label 不一定能够代表同桶内所有其他 label。一个局部分数稍低但 $T_{\max}$ 更小、跳数更少的前缀，可能在接入后续链路后重新胜出。因此，本文将 Sliced Dijkstra 定位为降低前缀陷阱概率的多标签基线，而不是具有最优性保证的精确算法。若后续实验需要报告“切片增强算法不低于单标签基线”的工程结果，可以在外层加入 baseline fallback：同时运行单标签 Dijkstra 与切片算法，并用同一个 BottleneckFidelityRoutingMetric 重新评估两条完整路径，最终返回预测保真度更高者。

3.4 证明与实现对照

当前代码中的 BottleneckFidelityRoutingMetric 已经把证明中的关键量都显式暴露出来了：

- `t_max_ms` 对应路径的时间瓶颈；
- `predicted_fidelity` 对应路径的当前打分；
- `IsBetter` 先比较 `predicted_fidelity`，再比较 `t_max_ms`，最后比较路径长度；
- `Dominates` 则要求在保真度和时间瓶颈上同时不劣，且至少一项严格更好。

因此，论文里的理论分析和代码实现是一致的：单标签 Dijkstra 只能作为基线，而真正有能力保留潜在反转路径的，是按 $T_{\max}$ 保留多个 slice 的多标签搜索。

3.5 实验设计占位

本章暂不报告最终实验数值，只保留后续验证所需的设计占位。后续实验将重点检查以下问题：

1. 在手工构造的前缀陷阱拓扑上，单标签 Dijkstra 是否会在中间节点过早剪枝；
2. Sliced Dijkstra 是否能保留不同 $T_{\max}$ 切片，从而恢复被单标签丢掉的候选；
3. 在真实拓扑和随机拓扑上，切片宽度 BucketWidthMs 与保留上限 K 如何影响路由结果；
4. 最终选路是否与混态仿真得到的端到端保真度一致。

这些结果将在后续实验章节中补充。当前版本先给出理论模型、算法语义与证明骨架，以保证后续实验叙述不会偏离代码实现。

Part II

量子网络模拟器实现

第 4 章 量子-经典混合仿真平台 qns-3 的架构设计

4.1 总体设计目标与分层结构

量子网络路由协议的研究对仿真平台提出了双重要求：一方面，需要以密度矩阵形式精确模拟量子比特在噪声信道和有损存储器中的演化，以获得可靠的保真度数据；另一方面，需要将量子协议置于真实的经典网络环境中运行，以捕捉因队列时延、信道拥塞和丢包等因素对量子业务产生的影响。现有量子网络仿真器（NetSquid、SeQUeNCe 等）内部维护独立的事件调度引擎，难以与经典网络协议栈在事件级别实现精确的时间同步，无法满足上述第二项要求。

qns-3 (Quantum ns-3) 是清华大学量子软件研究中心开发的量子网络仿真平台，其核心设计思路是将量子网络的全部事件调度纳入工业级网络仿真器 NS-3 Riley et al. (2010) 的统一离散事件时间轴，而非构建独立的仿真引擎。这一决策使得量子纠缠的生成、交换与测量等物理层事件，能够与 NS-3 中已高度成熟的 IP 路由、TCP 传输、CSMA/点对点链路等经典协议模型在同一时间轴上竞争调度，实现事件级别的精确同步。

qns-3 采用五层协议栈架构（图 4.1），自下而上依次为：ExaTN 张量网络计算后端、量子物理层、量子链路层、量子网络层，以及量子应用层。本工作开展之前，qns-3 仅具备底部两层（ExaTN 后端与量子物理层）及顶层的若干点对点应用协议；**量子链路层与量子网络层（含路由协议）均为本工作新实现的内容**，是填补平台协议栈中层缺失的核心工程贡献。各层之间通过清晰定义的 C++ 接口和 ns-3 回调机制耦合，整体遵循 ns-3 的模块化设计规范。

4.2 离散事件驱动的量子-经典协同仿真

qns-3 与其他量子网络仿真器最本质的区别在于其事件调度机制。在 NetSquid 等平台中，量子协议的时序逻辑由平台内部的事件引擎驱动，经典网络行为（如信令时延）往往以参数化的固定延迟注入，而非通过真实的协议栈产生。这导致两类时间轴的对齐仅为近似处理，无法捕捉真实的动态效应。

qns-3 的解决方案是将所有量子操作均封装为标准的 ns-3 事件，通过 `Simulator::Schedule()` 接口注册到全局事件队列。当仿真时间推进到某量子事件的触发时刻，事件回调函数调用 ExaTN 后端执行相应的张量网络计算；完成后，该回调函数可进一步向事件队列注册后续量子事件或触发经典报文的发



图 4.1 qns-3 量子网络仿真平台五层协议栈架构

送。以纠缠交换协议为例，混合事件流的典型执行序列如下：

1. 仿真时刻 $t = 0$ ：中继节点完成 Bell 态测量 (BSM)，测量结果 $\{m_L, m_R\}$ 通过真实的 ns-3 UDP 套接字打包为数据包，经由挂载了传播时延属性的 PointToPointChannel 发往枢轴节点；
2. 仿真时刻 $t = \delta$ ： δ 为 P2P 信道的真实传播时延（可含排队延迟和抖动），枢轴节点的 UDP 接收回调触发，从中解出测量结果；
3. 此时 `Simulator::Now()` 返回 δ ，物理层的退相干模型以 δ 为等待时间自动计算枢轴量子比特的保真度衰减；
4. 枢轴节点完成纠正操作后，继续向下游发送经典信令，触发链式的纠缠交换流程。

这一机制确保了“经典信令时延对量子态退相干的影响”能够被精确建模，而非依赖手工参数化的近似值。

4.3 ExaTN 密度矩阵计算后端

量子态的精确模拟由 `QuantumNetworkSimulator` 类负责，该类封装了 ExaTN (Exascale Tensor Networks) Lyakh (2022) 张量网络库，以密度矩阵表示全网量子态。

在内部实现中，每个量子比特对应张量网络 $\mathcal{N}$ 中的一个指标 (index)。 n 量子比特的系统密度矩阵 $\rho \in \mathbb{C}^{2^n \times 2^n}$ 被分解为“ket 半网络”与“bra 半网络”的张量积形式：

$$\rho = \sum_{\alpha} \lambda_{\alpha} |\psi_{\alpha}\rangle \langle \psi_{\alpha}| \quad (4.1)$$

其中每个 $|\psi_\alpha\rangle$ 对应张量网络中的一条收缩路径。

QuantumNetworkSimulator 维护了从量子比特名称到张量编号的双向映射 (ket 侧与 bra 侧各一份), 使得上层物理层接口可通过量子比特的字符串名称 (如 "A_pL"、"B_src_q") 定位并操作对应的密度矩阵张量。保真度计算通过完整的密度矩阵收缩完成, 绕过了依赖解析公式的近似计算:

$$F = \langle \Phi^+ | \rho_{AB} | \Phi^+ \rangle \quad (4.2)$$

其中 $|\Phi^+\rangle$ 为目标 Bell 态。

4.4 量子物理层与存储器退相干模型

量子物理层由 QuantumPhyEntity 类实现, 负责将上层的量子操作请求转换为对 ExaTN 后端的具体调用, 并管理访问控制 (防止不同网络节点交叉操作对方的量子比特)。其主要功能包括: 量子比特的纯态/混态初始化 (GenerateQubitsPure/GenerateQubitsMixed)、量子门施加 (ApplyGate)、Bell 态测量 (BellMeasure) 及保真度计算 (CalculateFidelity)。

退相干效应由 QuantumMemory 类统一管理。该类为每个存储的量子比特记录一个时间戳 t_{last} (类型为 ns-3 的 Time 对象), 在任何操作施加于该量子比特之前, 系统首先计算等待时间 $\Delta t = t_{\text{now}} - t_{\text{last}}$, 并调用 TimeModel 对单量子比特施加相位翻转信道:

$$\mathcal{E}_\lambda(\rho) = \frac{1+\lambda}{2}\rho + \frac{1-\lambda}{2}(Z\rho Z), \quad \lambda = e^{-\Delta t/\tau}, \quad (4.3)$$

其中 τ 为该量子比特所对应节点的相干时间参数。若一对以 Werner 态初始化的 Bell 对, 其左右两端分别等待 $\Delta t_L, \Delta t_R$, 则当前 Bell 保真度应写为

$$F(\Delta t_L, \Delta t_R) = \frac{2F^{(0)} + 1 + e^{-(\Delta t_L/\tau_L + \Delta t_R/\tau_R)}(4F^{(0)} - 1)}{6}, \quad (4.4)$$

而不是把上一时刻的 Bell 保真度直接乘上 $e^{-\Delta t/\tau}$ 。这一“惰性求值” (lazy evaluation) 策略确保了退相干操作与 ns-3 仿真时钟严格同步, 无需独立的退相干事件定时器。

qns-3 实现了三类量子错误模型:

- **TimeModel**: 基于时间的退相干模型, 如式 (4.3), 由 QuantumMemory 在操作前自动调用;
- **DephaseModel**: 相位翻转噪声, 模拟量子门操作或信道传输引入的去相位误差;
- **DepolarModel**: 去极化信道噪声, 将量子态以概率 $1-p$ 映射为最大混合态,

默认信道保真度参数 $p = 0.95$ 。

4.5 量子链路层：纠缠对分发与状态管理

量子链路层 (QuantumLinkLayerService) 负责在相邻节点间建立 Bell 纠缠对，是网络层建立多跳端到端纠缠的基础原语。链路层屏蔽了纠缠生成的概率性失败细节，向上层提供以下接口语义：向指定邻居节点发出纠缠请求，并在成功后通过 ns-3 回调通知上层 EntanglementId、所生成的量子比特名称及初始保真度。

每条成功建立的纠缠对封装为 EntanglementInfo 结构体，包含本地和远端量子比特名称、生成时刻 (createdAt, 类型为 Time) 及有效性标志。链路层的状态机仅含四种状态：PENDING (等待)、READY (可用)、CONSUMED (已被上层消费)、FAILED (失败)，状态转移通过 ns-3 事件调度触发，全程在统一时间轴上推进。

4.6 量子网络层与路由协议

QuantumNetworkLayer 类实现量子网络层 (L3) 逻辑，管理多跳纠缠路径的建立与生命周期。其核心抽象是路径 (Path)，每条路径具有唯一标识符 PathId，并经历如下生命周期：

$$\text{PENDING} \longrightarrow \text{READY} \longrightarrow \begin{cases} \text{CONSUMED} \\ \text{FAILED} \end{cases} \quad (4.5)$$

网络层负责协调每一跳链路层纠缠的建立、纠缠交换操作 (EntanglementSwap) 的触发时机，以及最终端到端纠缠对的保真度检验。路径建立成功 (READY) 后，通过注册的回调函数通知上层应用，应用方可凭 PathId 消费 (消耗) 该纠缠资源。

qns-3 目前实现了两种路由协议，均继承自抽象基类 QuantumRoutingProtocol：

1. **DijkstraRoutingProtocol**: 单标签 best-first 基线，每个节点只保留一条 incumbent label，适合作为当前非保序度量下的 $K = 1$ 对照组；
2. **SlicedDijkstraRoutingProtocol**: 多标签切片搜索，按 predicted_fidelity 和 t_max_ms 同时比较候选路径，并可通过 BucketWidthMs 在 $T_{\max}$ 轴上保留多个切片。

值得注意的是，当前实现中的 BottleneckFidelityRoutingMetric

不满足路由代数中的保序性 (isotonicity) 条件Wang et al. (2025), 即路径拼接后的度量大小关系不保持单调性。这一固有缺陷是本文第 3 章所研究的“路由非保序性”问题的理论根源。

4.7 经典信令信道的建模方式

量子网络协议中的经典信令 (Bell 态测量结果通知、纠正操作指令、路由控制报文等) 需要在节点间可靠传递, 其传递时延直接决定了枢轴节点量子比特的等待时间, 进而影响退相干损耗。

qns-3 在应用层协议 (如 TelepAdaptApp、DistillNestedAdaptApp) 中通过 IPv6 UDP 套接字承载上述经典信令, 由此获得了以下仿真保真性保证:

- **真实传播时延:** 数据包经由 ns-3 PointToPointChannel 或 CsmaChannel 传输, 传播时延由信道的 Delay 属性决定, 而非手工注入的固定延迟;
- **排队时延与拥塞:** 链路的数据速率属性决定了串行化时延; 当量子信令报文与经典背景流量 (如 CBR 流) 竞争共享信道时, 排队时延自然涌现;
- **随机丢包:** 可在接收侧的 NetDevice 上挂载 RateErrorModel, 以指定的每包丢弃概率模拟信道丢包, 测试量子协议在经典信令丢失时的鲁棒性;
- **时延抖动:** 可在发送侧引入均匀随机抖动, 模拟实际链路中的时延变化。

以上特性使得在 qns-3 中运行的量子路由协议能够被置于接近真实的量子-经典混合网络环境中进行压测。这也是 qns-3 区别于其他量子网络仿真工具的核心工程价值: 量子协议的正确性不仅在理想信道假设下得到验证, 同时在具有真实统计特性的经典网络动态下得到评估。

参考文献

- LIN H, DENG R, YAO C Z, et al. Control flow adaption: An efficient simulation method for noisy quantum networks[A/OL]. 2024. arXiv: 2412.08956. <https://arxiv.org/abs/2412.08956>.
- CALEFFI M. Optimal routing for quantum networks[J]. IEEE Access, 2017, 5: 22299-22312.
- ALI I, WAHAB A, MAHMOOD Z. Optimal quantum network coding for quantum communication [J]. IEEE Communications Letters, 2017, 21(7): 1673-1676.
- WANG Z, et al. KOP: K-optimal paths routing for quantum networks via routing algebra[A]. 2025.
- SUTCLIFFE L, et al. Fidelity-aware multi-path routing for multipartite entanglement distribution[A]. 2025.
- SHI S, QIAN C. Concurrent entanglement routing for quantum networks: Model and designs[C/OL]// Proceedings of the ACM SIGCOMM 2020 Conference. 2020: 62-75. DOI: 10.1145/3387514.3405853.
- GU H, YU R, LI Z, et al. QuESat: Satellite-assisted quantum internet for global-scale entanglement distribution[C]//IEEE INFOCOM. 2025.
- RILEY G F, HENDERSON T R. The ns-3 network simulator[C]//WEHRLE K, GUNES M, GROSS J. Modeling and Tools for Network Simulation. Berlin, Heidelberg: Springer, 2010: 15-34.
- LYAKH D I. ExaTN: Exascale tensor networks[A]. 2022.